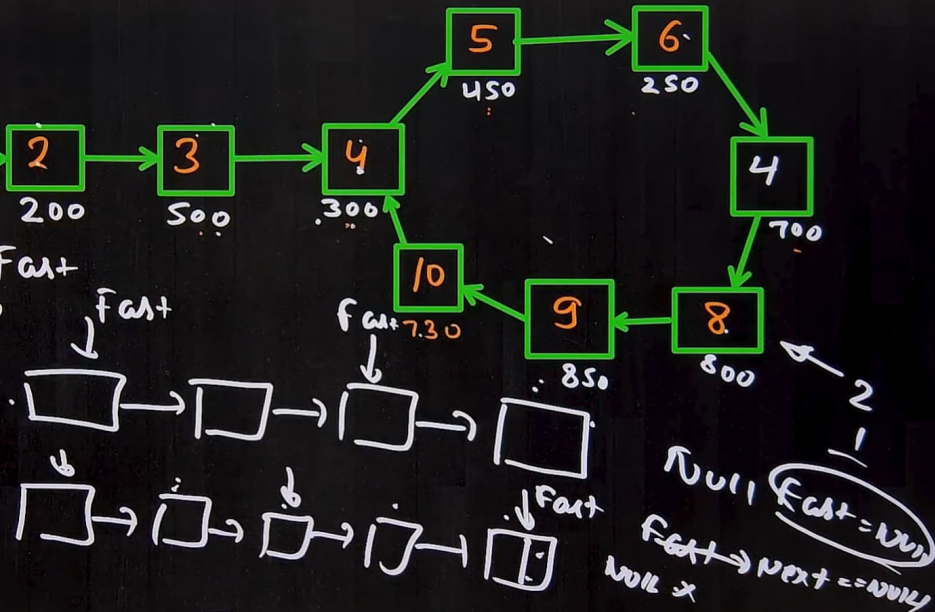


Detect Loop in Linked list

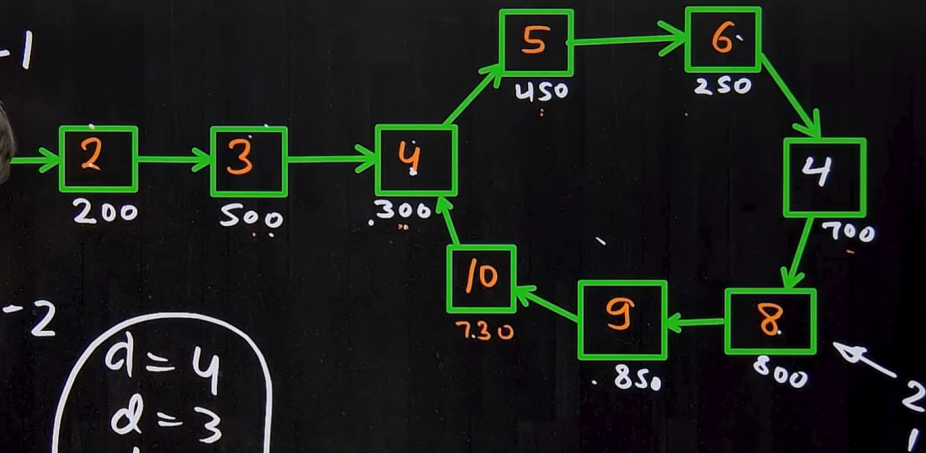
slow
-1

Fast
-2



Detect Loop in Linked list

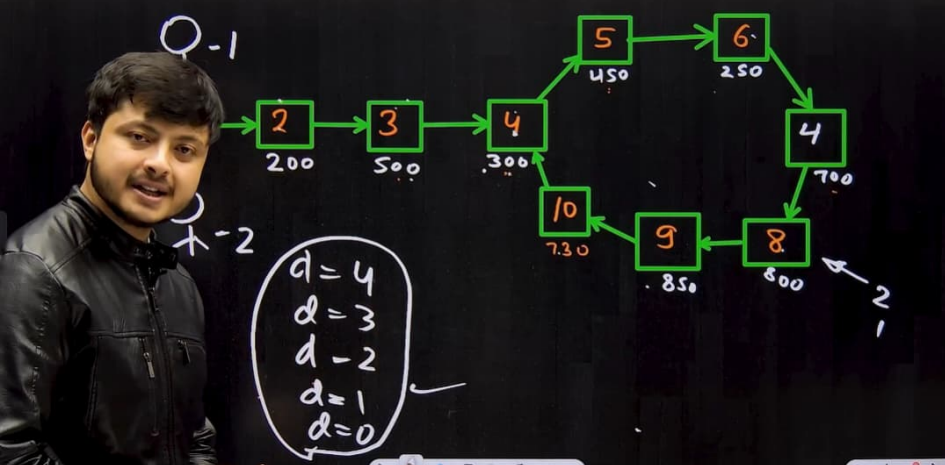
Q-1



Q-2

$d=4$
 $d=3$
 $d=2$
 $d=1$
 $d=0$

Detect Loop in Linked list



Replies



P

@ParvezKhan-mj7yt

when fast pointer running at 3 and slow at 1, n is the number of nodes in cycle, if n is odd then for sure it will meet at some point, but if n is even they may meet at first cycle, and if they don't meet at first cycle, they will never meet for sure, hence we can't use it to detect cycle

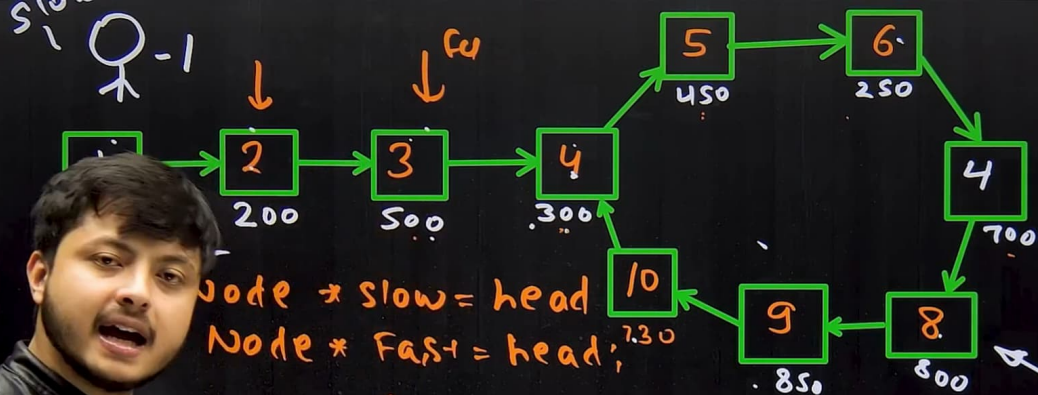


3



Reply...





Node * slow = head;
Node * fast = head;

```

while (fast != NULL && fast->next != NULL)
{
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast)
        return 1;
}
  
```



<> Problem

Editorial

Submissions

Comments

Detect Loop in linked list

Easy Accuracy: 43.49% Submissions: 394K+ Points: 2

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given a linked list of N nodes. The task is to check if the linked list has a loop. Linked list can contain self loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

x(position at which tail is connected) = 2

Output: True

Explanation: In above test case N = 3.

The linked list with nodes N = 3 is given. Then value of x=2 is given which means last node is connected with xth node of linked list. Therefore, there exists a loop.

Example 2:

Input:

C++ (g++ 5.4)

Average Time: 20m

Start Timer

```
// FUNCTION TO CHECK IF THE LINKED LIST HAS A LOOP.
50 bool detectLoop(Node* head)
51 {
52     // your code here
53     Node *slow = head, *fast = head;
54
55     while(fast!=NULL&&fast->next!=NULL)
56     {
57         slow = slow->next;
58         fast = fast->next->next;
59
60         if(slow==fast)
61             return 1;
62     }
63
64     return 0;
65
66 }
67
68 };
69
70 // } Driver Code Ends
```

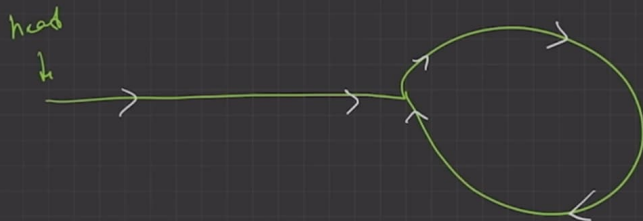


Custom Input

Compile & Run

Submit

{Three
90
Challen

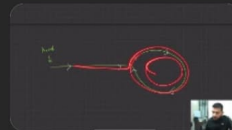


In this video



Chapters

Transcript



why this works ?

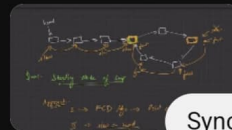


21:31



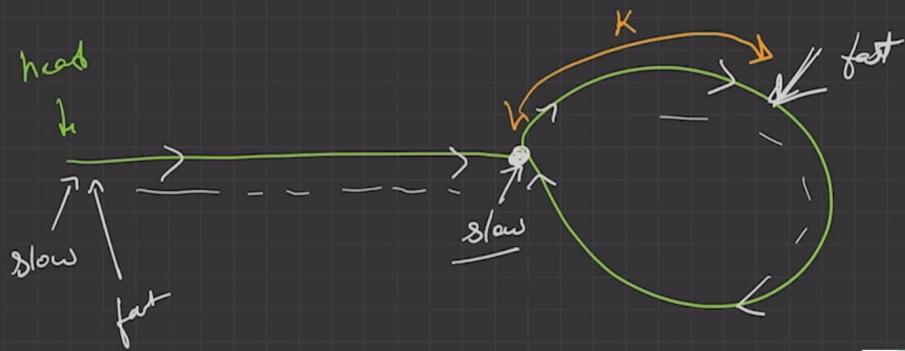
Approach #3 - find
starting node of Loop

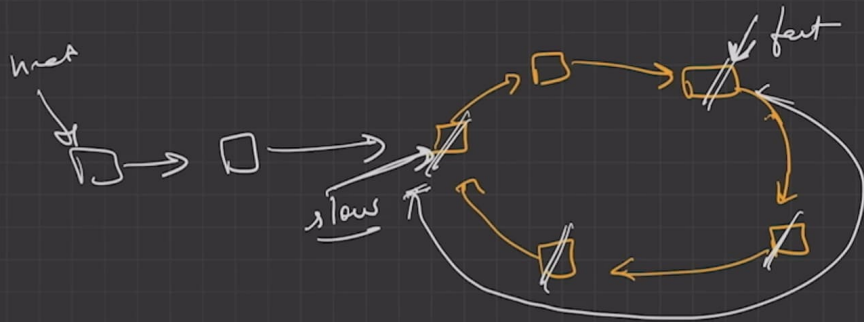
25:30



Proof - why this
works ?

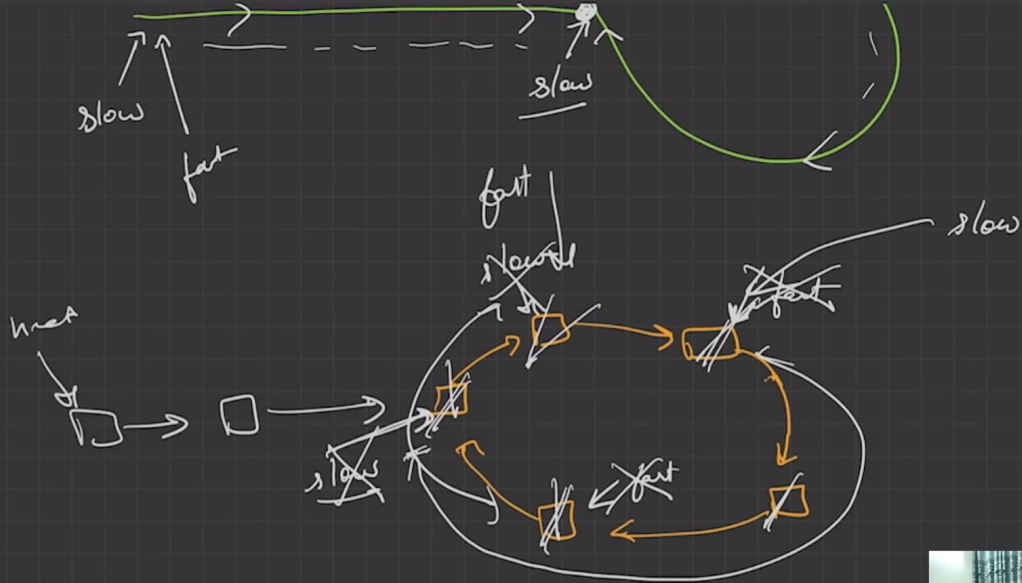
Sync to video





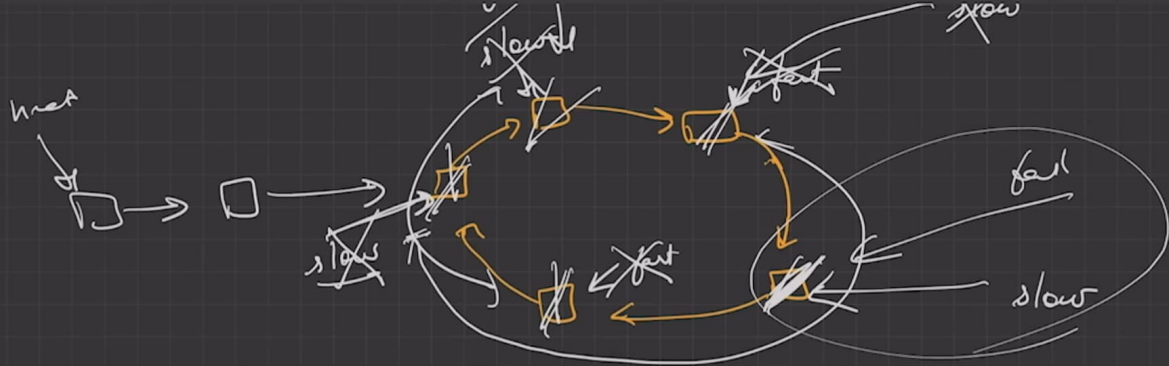
Distance: 4





Distance: 4, 3, 2

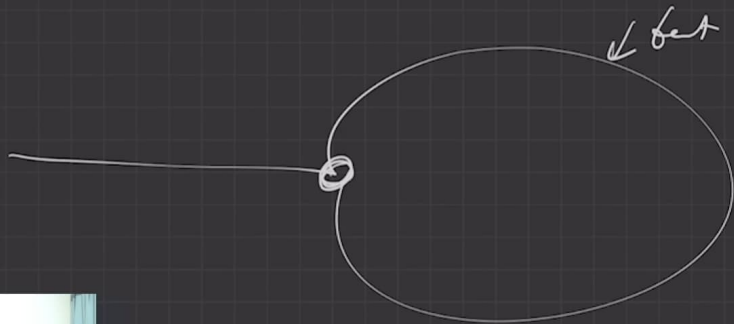




Distance: 4, 3, 2, 1



Distance: 4, 3, 2, 1



iteration



distance



-1

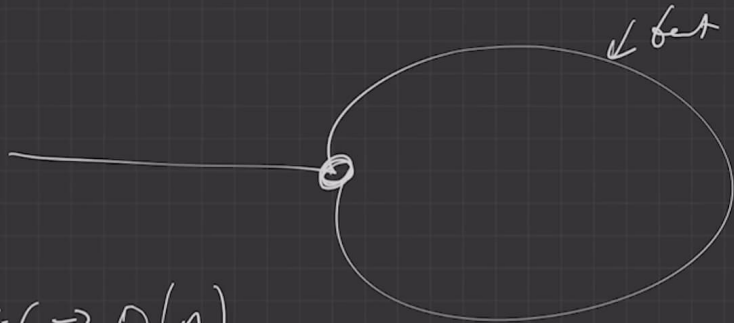


at one point

meet

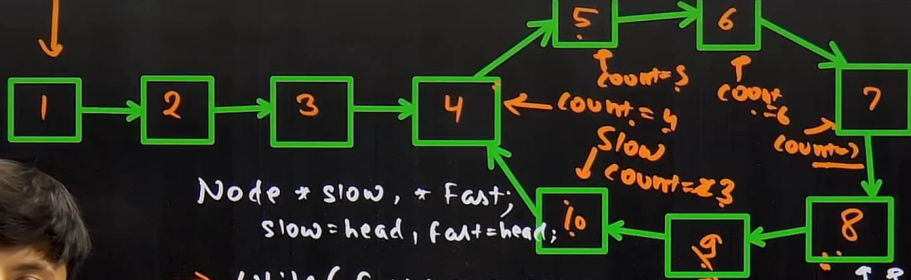


T.C $\rightarrow O(n)$
S.C $\rightarrow \underline{\underline{O(1)}}$



iteration
↓
distance
↓
-1
↙
at one point
meet ←





Node * slow, * fast;
slow = head, fast = head;

→ while (fast != NULL && fast->next != NULL)

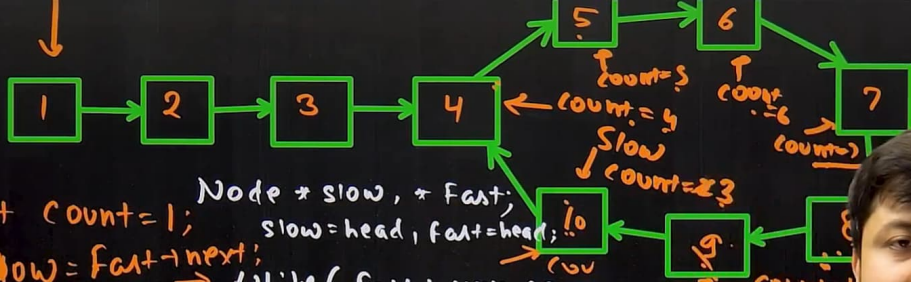
slow = slow->next;

fast = fast->next->next;

if (slow == fast)

break;

if (fast == NULL || fast->next == NULL)
return 0;



```

Node * slow, * fast;
int count = 1;
slow = head, fast = head;
slow = fast->next;
while (fast != NULL && fast->next != NULL)
{
    slow = slow->next;
    fast = fast->next->next;
    count++;
    if (slow == fast)
    {
        slow = slow->next; break;
    }
}
return count;
if (fast == NULL || fast->next == NULL)
    return 0;
  
```